

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄静怡 学号 2024K8009907042 专业 计算机科学与技术
实验项目编号 2 实验名称 简单功能型处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支 (具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图,可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF,再插入到 L^AT_EX 中)

1 单周期 MIPS 处理器设计

本次单周期处理器设计将整体 CPU 架构划分为 ALU、移位器 Shifter、译码模块 ID、执行模块 EX、程序计数器 PC 以及顶层主程序六大功能部分。

其中算术逻辑单元 ALU 已在前期基础实验中完成主体功能开发, 本实验仅在原有运算逻辑上新增 XOR、NOR、SLTU 三种运算操作即可, 无需重构整体结构。本次设计开发重点主要放在移位器 Shifter 与简易单周期 CPU 整体架构的编写与联调上。

初期设计曾考虑将完整 Simple-cpu 逻辑全部写在同一个文件中, 但发现全部逻辑集中在单一文件会造成代码体量过大、结构混乱, 不利于阅读、调试与功能划分。因此改为按照处理器五级流水线思想拆分逻辑, 依据取指、译码、执行、访存、写回以及跳转、PC 计数等工作阶段, 把整体功能拆解为多个独立子模块。

按照功能拆分后, 新增 EX 执行模块, 负责运算数据预处理、调用 ALU 与移位器完成运算, 并进行分支条件判断、确定是否执行跳转; ID 译码模块单独负责指令解析、字段拆分, 并依据指令类型生成整机所有控制信号; PC 模块专门完成程序地址自增计数、分支跳转地址刷新等功能。

同时额外单独建立 define 宏定义文件, 统一记录并管理所有指令操作码、功能码及控制信号的二进制编码常量, 实现参数集中管理, 提高代码可维护性与可读性。整体采用模块化分层设计, CPU 运行时按取指、译码、执行、跳转、计数等流程逐级调用对应子模块, 协同完成单周期指令的完整执行流程。

2 关键模块设计

shifter: 移位器模块是本次实验的重点设计模块之一, 主要功能是接收译码模块 (ID) 输出的移位控制信号 (Shiftop-opAB) 和操作数, 完成指定的移位操作, 为执行模块 (EX) 提供移位运算结果。该模块支持常用的移位类

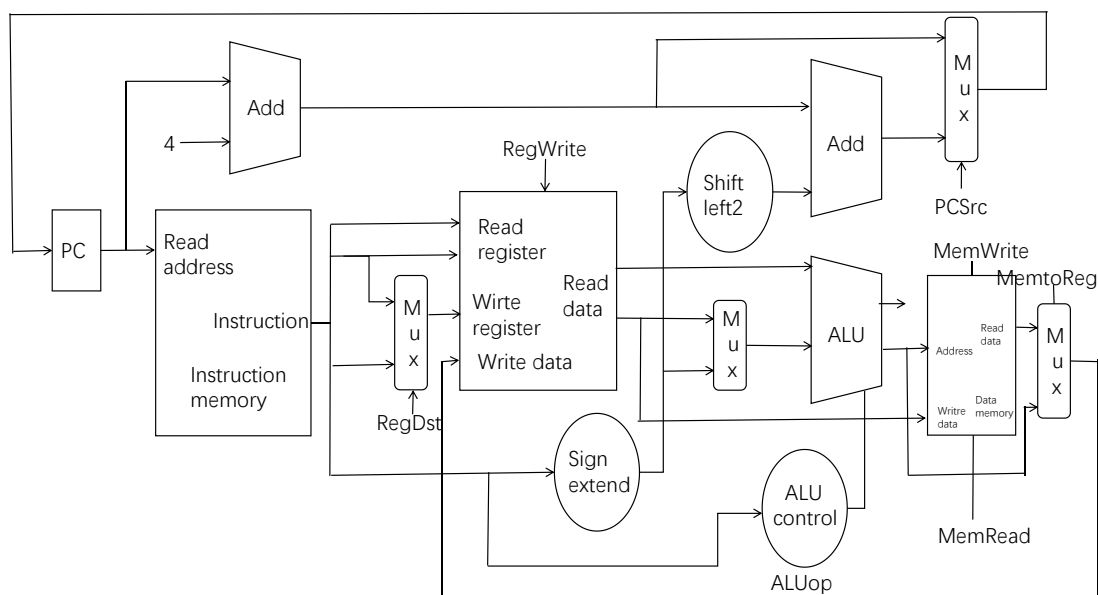


图 1: cpu 逻辑电路图

型,包括逻辑左移(SLL)、逻辑右移(SRL)和算术右移(SRA),移位位数由输入的操作数字段决定,运算过程中严格遵循移位操作的逻辑规则,确保移位结果的准确性。移位器的输出直接传入执行模块,与 ALU 运算结果协同,完成指令规定的运算功能,其性能直接影响 CPU 整体的运算效率和指令执行正确性。

id:译码模块是 CPU 的核心控制模块之一,其功能至关重要,因为该阶段会产生整机运行所需的几乎所有控制信号,同时解析指令字段、生成 ALU 和 shifter 模块所需的两个操作数(ALU-opAB 和 Shiftop-opAB)。该模块接收程序计数器(PC)传入的指令,首先对指令的操作码、功能码进行解析,判断指令类型(如 R 型、I 型、分支指令等),随后拆分指令中的寄存器地址、立即数等关键字段,读取对应寄存器的值作为操作数,分别传入 ALU 和移位器。同时,根据指令类型生成对应的控制信号,包括 ALU 运算控制信号、移位控制信号、分支跳转控制信号、访存控制信号等,为后续执行、访存等阶段提供精准的控制依据,确保各模块协同工作。

ex:执行模块是 CPU 的运算核心,负责整合 ALU 和移位器的运算功能,同时完成分支跳转条件的判断。该模块接收译码模块(ID)输出的控制信号、ALU 操作数(ALU-opAB)、移位器操作数(Shiftop-opAB),根据控制信号的指示,选择性调用 ALU 或移位器完成对应运算:当指令为算术逻辑运算时,调用 ALU 执行运算(含新增的 XOR、NOR、SLTU 三种操作);当指令为移位运算时,调用移位器执行移位操作。此外,执行模块还会对分支指令的跳转条件进行判断,结合运算结果生成分支跳转信号,传入 PC 模块,决定 PC 地址是否需要跳转,确保指令流的正确执行。

pc:程序计数器(PC)模块主要负责指令地址的计数与刷新,是 CPU 取指阶段的核心模块。该模块接收时钟信号(clk)、复位信号(rst-n)以及执行模块(EX)传入的分支跳转信号,正常情况下,PC 地址按固定步长(通常为 4)自增,指向当前执行指令的下一条指令地址,确保指令的顺序执行;当接收到执行模块传来的分支跳转信号时,PC 地址会立即刷新为跳转目标地址,实现指令的跳转执行。同时,PC 模块输出当前的指令地址,传入指令存储器,用于读取下一条待执行的指令,为整个 CPU 的指令流提供地址支撑。

simple-cpu: simple-cpu 模块作为整个单周期 CPU 的顶层模块,核心功能是整合上述所有子模块(shifter、ID、EX、PC、ALU 等),实现各模块的信号连接与协同工作,完成单周期指令的完整执行流程。该模块不再将所有逻辑集中在单一文件中,而是按照取指、译码、执行、跳转、计数的流程,通过模块例化的方式,将各子模块连接为一个有机整体:首先通过 PC 模块获取指令地址,读取指令后传入 ID 模块进行译码,译码生成的控制信号和操作数分别传入 EX、ALU、shifter 模块,EX 模块调用 ALU 和 shifter 完成运算并判断跳转,最终将跳转信号反馈给 PC 模块,实现指令的循环执行。此外,顶层模块还集成了指令存储器和数据存储器,负责指令的存储与数据的读

写,确保 CPU 正常运行所需的数据和指令供应。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 译码

1.1 与内存读写有关的控制信号

第一部分包括内存读信号,内存写信号,内存向寄存器写信号,内存读出数据以后的符号位扩展选择信号(零扩展和符号位扩展),以及读出的内存长度(包括字节,半字,字)

```
//内存控制信号和内存->寄存器
assign MemRead =opcode[5]&~opcode[3];
assign MemWrite =opcode[5]&opcode[3];
assign MemtoReg =MemRead;
assign Ext =(opcode[5:4]==2'b10&&(opcode[2]==0))? 1: //符号扩展
          0; //零扩展
assign memLen= ({opcode[5:4],opcode[1:0]}==4'b1000)? 2'b00://字节
               ({opcode[5:4],opcode[1:0]}==4'b1001)? 2'b01://半字
               ({opcode[5:4],opcode[1:0]}==4'b1010)? 2'b10:
               //低两位为10的时候是非对齐,用10来标记非对齐
               ({opcode[5:4],opcode[1:0]}==4'b1011)? 2'b11://字
               2'b00;//默认
```

其中内存读写信号由 opcode 直接产生,发现读操作的各个指令(包括各种 L 型指令)中,内存读一定伴随着写入寄存器,于是直接将 MemtoReg 赋值为 MemRead,Ext 的零扩展主要面对的是读出的数据进行无符号写入的情况(LBU,LHU),Memlen 的产生主要是面对根据 opcode 的高两位(用于确定是内存读写命令)和低两位(用于确定是字节,半字,字以及非对齐,我们这里利用 memlen=10 来标记非对齐的情况)来进行判断。

第二部分是读写部分的地址选择,以及非对齐读写的内容和写入位置的选择,本次实验采用小尾端设计。

1.Address,用于确定读写命令的地址

```
assign Address= ex_Result & 32'hfffffff; //对内存访存的地址
wire [1:0] effaddr_ctrl;
assign effaddr_ctrl=ex_Result[1:0];
```

由于有非对齐存储的情况,于是将 Address 的低两位清零,来将这两位作为控制信号,通过改变写入的内容,来实现不同位置的读写以及非对齐读写。

2. 有关内存读写的各个控制信号,包括左右非对齐读写控制信号,另外在本实验中我们将读和写的内容分为 4 个字节来进行操作,极大程度上提高了代码可读性和简洁度

```
wire [1:0] effaddr_ctrl;
assign effaddr_ctrl = ex_Result[1:0];
wire rl_sel; //用于选择左非对齐读入和右非对齐写入,0为L,1为R
assign rl_sel = opcode[2];
//将从内存读出来的4个字节数据分成4个字节,用于小尾端和非对齐写入
wire [7:0] read_byte_3;
wire [7:0] read_byte_2;
wire [7:0] read_byte_1;
```

```

wire [7:0] read_byte_0;
assign read_byte_3=Read_data[31:24];
assign read_byte_2=Read_data[23:16];
assign read_byte_1=Read_data[15:8];
assign read_byte_0=Read_data[7:0];
//用于记录写回寄存器原本的数据,用于生成RF_wdata
wire [31:0] wdata_i;
assign wdata_i = rdata2;
//由于非对齐和对齐中写入的字节不同,于是更改Write_data;
wire [5:0] Write_data_sel;
assign Write_data_sel={rl_sel,memLen,effaddr_ctrl};
//将从寄存器读出来的4个字节数据分成4个字节,用于小尾端和非对齐写入
wire [7:0] wb3;
wire [7:0] wb2;
wire [7:0] wb1;
wire [7:0] wb0;
assign wb3 = rdata2 [31:24];
assign wb2 = rdata2 [23:16];
assign wb1 = rdata2 [15: 8];
assign wb0 = rdata2 [ 7: 0];

```

3. Write-strb 代表写入的位置, 对于不同的读写写命令有着不同的赋值, 利用左右非对齐写入和地两位来产生非对齐的控制信号

```

//对写内存进行strb的赋值, 是4位信号: [3][2][1][0], 每一位对应内存4字节中的一个字节, 1 =写这个字节, 0=不写, 保持原值
assign Write_strb = //写字节, 用effaddr的最后两位来控制写的位置
(memLen==2'b00)? {(effaddr_ctrl==2'b11),(effaddr_ctrl==2'b10),(effaddr_ctrl==2'b01),(effaddr_ctrl==2'b00)}:
//写半字, 10时为1100, 01时为0011
(memLen==2'b01)? {effaddr_ctrl[1],effaddr_ctrl[1],~effaddr_ctrl[1],~effaddr_ctrl[1]}:
//SWL
({rl_sel,memLen}==3'b010)? ((effaddr_ctrl==2'b00)? 4'b0001:
(effaddr_ctrl==2'b01)? 4'b0011:
(effaddr_ctrl==2'b10)? 4'b0111:
4'b1111):
//SWR
({rl_sel,memLen}==3'b110)? ((effaddr_ctrl==2'b00)? 4'b1111:
(effaddr_ctrl==2'b01)? 4'b1110:
(effaddr_ctrl==2'b10)? 4'b1100:
4'b1000):
//表示写整个字
4'b1111;

```

4. RF-wdata 代表写入寄存器的数据, 由于在本实验中我们将读和写的内容分为 4 个字节来进行操作, 对齐处理的情况比较简单, 但是在非对齐的情况下, 我们需要利用地址的低两位作为控制信号来确定写入的内容 (根据 MIPS) 指令集手册, 值得一提的是利用 00 和 10 来分别表示半字, Write-data 同理。

5. Write-data 表示写入内存的数据, 要注意符号拓展和零扩展, 以及左右非对齐情况下的写入

1.2 寄存器输入的控制信号

主要是控制读或者写的内容是 rt 寄存器还是 rd 寄存器

```

assign RegDst = (opcode==`R_TYPE)? 1:0; //控制寄存器写地址选择, 0就用rt, 1就用rd

```

1.3 分支和跳转信号

分支和跳转信号以及 R 型的跳转指令, 利用 opcode 和 func 来判断他们, 利用布尔表达式产生控制信号

```

assign Branch = (opcode[5:2]==4'b0001 || opcode==`REGIMM);

```

```
//assign RF_wdata=(opcode==JAL || (opcode==R_TYPE && func==JALR))?(PC+S://对于JALR指令和I型计算指令，将PC+8写回
//      (opcode==R_TYPE || opcode[5:3]==3'b001)?ex_Result://直接把ALU/移位器结果ex_Result 写回寄存器
//对于读字节的操作，其中第二个信号用于控制读入的字节在4个字节中位于哪个位置
    (memLen==2'b00)?((effaddr_ctrl==2'b00)? ((Ext==1)? {{24{read_byte_0[7]}},read_byte_0}: {{24{1'b0}},read_byte_0}):
        (effaddr_ctrl==2'b01)? ((Ext==1)? {{24{read_byte_1[7]}},read_byte_1}: {{24{1'b0}},read_byte_1}):
        (effaddr_ctrl==2'b10)? ((Ext==1)? {{24{read_byte_2[7]}},read_byte_2}: {{24{1'b0}},read_byte_2}):
        ((Ext==1)? {{24{read_byte_3[7]}},read_byte_3}: {{24{1'b0}},read_byte_3}))
    ):
//对于对齐读半字的操作
    (memLen==2'b01)?((effaddr_ctrl==2'b00)? ((Ext==1)? {{16{read_byte_1[7]}},read_byte_1,read_byte_0}: {{16{1'b0}},read_byte_1,read_byte_0}):
        ((Ext==1)? {{16{read_byte_3[7]}},read_byte_3,read_byte_2}: {{16{1'b0}},read_byte_3,read_byte_2})):
    ):
//LWL非对齐左读入的情况，以下四种非对齐写入的情况
    ({r1_sel,memLen)==3'b010)? ((effaddr_ctrl==2'b00)? {read_byte_0,wdata_i[23:0]}:
        (effaddr_ctrl==2'b01)? {read_byte_1,read_byte_0,wdata_i[15:0]}:
        (effaddr_ctrl==2'b10)? {read_byte_2,read_byte_1,read_byte_0,wdata_i[7:0]}:
        {read_byte_3,read_byte_2,read_byte_1,read_byte_0})
    ):
//LWR非对齐右读入的情况，以下四种非对齐写入的情况
    ({r1_sel,memLen)==3'b110)? ((effaddr_ctrl==2'b00)? {read_byte_3,read_byte_2,read_byte_1,read_byte_0}:
        (effaddr_ctrl==2'b01)? {wdata_i[31:24],read_byte_3,read_byte_2,read_byte_1}:
        (effaddr_ctrl==2'b10)? {wdata_i[31:16],read_byte_3,read_byte_2}:
        {wdata_i[31:8],read_byte_3}):
//表示读整个字的操作
    Read_data;

assign Write_data = (memLen==2'b00)?((effaddr_ctrl==2'b00)? {{24{1'b0}},wb0}:
    (effaddr_ctrl==2'b01)? {{16{1'b0}},wb0,{8{1'b0}}}:
    (effaddr_ctrl==2'b10)? {{8{1'b0}},wb0,{16{1'b0}}}:
    {wb0,{24{1'b0}}}):
    (memLen==2'b01)? ((effaddr_ctrl[1]==0)? {{16{1'b0}},wb1,wb0}:
        {wb1,wb0,{16{1'b0}}}):
    //SWL 从高位往低位写：写 wb3/wb3+wb2/wb3+wb2+wb1
    (Write_data_sel==5'b01000)? {{24{1'b0}},wb3}:
    (Write_data_sel==5'b01001)? {{16{1'b0}},wb3,wb2}:
    (Write_data_sel==5'b01010)? {{8{1'b0}},wb3,wb2,wb1}:
    //SWR 从低位往高位写：写 wb0 /wb1+wb0 /wb2+wb1+wb0
    (Write_data_sel==5'b11001)? {wb2,wb1,wb0,{8{1'b0}}}:
    (Write_data_sel==5'b11010)? {wb1,wb0,{16{1'b0}}}:
    (Write_data_sel==5'b11011)? {wb0,{24{1'b0}}}:
    rdata2;
```

```
assign Jump = (opcode[5:1]==5'b00001 || (opcode==`R_TYPE && func[3:1]==3'b100) || opcode==`JAL);
```

1.4 读写寄存器的赋值

raddr1 和 raddr2 就是 rs 和 rt, 值得注意的是 waddr 的赋值, 如果是 R 型指令需要赋值为 rd, 另外对于 JAL 指令, 需要赋值为 31, 因为要将 PC+8 赋值给该特殊寄存器, waddr 在其他情况下为 rt

```
//读寄存器地址
assign raddr1 = rs;
assign raddr2 = rt;
//写寄存器地址
assign waddr = (RegDst==1)? rd:
               (opcode==`JAL)? 5'b11111:
               rt;
```

1.5 运算器和移位器的功能选择信号

这两部分同样是本次实验的重点，运算器的部分我列出了各个指令以及他们所对应的 ALUop，并且利用卡诺图化简，来最终实现这个 ALUop 的赋值，另外我将不需要 ALU 的指令（包括跳转）的 ALUop 赋值为 OR，对应的 ALU 操作数和是 0，以避免意料之外的错误。对于移位操作器 Shifter，住的一体的就是 LUI 指令，在这个指令我们赋值为左移，另外有的情况下不移位，我们设置为 01

```
assign Shiftop = ((opcode==`R_TYPE && func[5:3]==3'b000))? func[1:0] : //Rtype算数移位指令
                ((opcode==`LUI))?      2'b00      :      //加载到高位
                2'b01;      //其他情况下不移位
```

```
// 运算器功能选择信号
assign ALUop = ((opcode[5]==1) || (opcode=="ADDIU") || (opcode=="R_TYPE" && (func=="ADDU" || func[5:3]==3'b001)))? `ADD : //用到add的情况
               ((opcode=="R_TYPE" && func=="SUBU") || opcode[5:1]==5'b00010)? `SUB : //用到sub的情况
               ((opcode=="R_TYPE" && func=="AND_") || opcode=="ANDI")? `AND : //用到and的情况
               ((opcode=="R_TYPE" && func=="OR_") || opcode=="ORI")? `OR : //用到or的情况
               ((opcode=="R_TYPE" && func=="XOR_") || opcode=="XORI")? `XOR : //用到xor的情况
               (opcode=="R_TYPE" && func=="NOR_")? `NOR : //用到nor的情况
               ((opcode=="R_TYPE" && func=="SLT_") || opcode=="SLTI" || opcode=="REGIMM" || opcode[5:1]==5'b00011)? `SLT : //用到SLT的情况
               ((opcode=="R_TYPE" && func=="SLTU_") || opcode=="SLTIU")? `SLTU : //用到SLTU的情况
               `OR; //有的情况不需要ALU,默认赋值为or
```

另外考虑到无论如何 ALU 和 Shifter 总会计算出一个结果, 这意味着每一条指令总会产生两个运算的结果, 于是我们引入一个新的控制信号, 用于在 ALU 和 Shifter 的结果之间进行选择

```
assign
    AluShi_sel=((opcode=="R_TYPE"&&func[5:3]==3'b000)||opcode=="LUI");//结果为1,选择shifter,否则选择alu
```

1.6 ALU 和 Shifter 操作数的选择

首先我们要对立即数进行拓展, 分为符号位拓展和零扩展

```
wire [31:0]op_imm;
assign op_imm = (opcode[5:2]==4'b0011)? {16{1'b0}},imm}: //零扩展
               {{16{imm[15]}},imm}; //符号扩展
```

其次是 ALU 的两个操作数, 另外对于 bgez 等四个指令, 我通过对操作数的选择来简化之后分支指令的产生, 例如 bgtz 是数据大于 0, 于是在这种情况下, 我将操作数 a 设为 0, 操作数 b 设为 rdata1, 这样就可以直接利用 SLT 的结果来判断, 省略了之后对 SLT 结果的一步操作, 具体的 ALUop A 和 ALUop B 如下

```
assign ALUop_A=(opcode=="BGTZ"? 32'b0: //0<rdata1时为1,表示rdata大于0时分支
                (opcode=="REGIMM"&&REG[0]==1)? {32{1'b1}}:rdata1;//-1<rdata1时为1,表示rdata大于等于0时分支

assign ALUop_B=(opcode=="R_TYPE"&&(func=="MOVZ" || func=="MOVN"))?
    32'b0://对于R中的两个移动指令,ALUop为add,因此rdata1+0
    (opcode=="REGIMM"&&REG[0]==0)? 32'b0: //rdata1小于0时分支
    (opcode=="BLEZ)? 32'b1: //rdata1小于1时分支,也就是小于等于0时分支
    (opcode=="BGTZ||(opcode=="REGIMM"&& REG[0]==1))? rdata1://对于上面的两种情况,B应该是rdata1
    (ALUSrc)? op_imm:rdata2;
```

之后是对 Shifter 操作数的赋值, 这个要简单很多, 在 R 型指令中利用 shamt 来进行赋值, 或者利用 rdata1 中的低 5 位数据来进行赋值 (可变移位), 同样还是将 LUI 单独拿出来判断

```
assign Shiftop_A=(opcode=="LUI)? op_imm: //LUI指令对立即数做移位
                rdata2; //rt移动
assign Shiftop_B= (opcode=="R_TYPE"&&rs==5'b0)? shamt:
    (opcode=="LUI)? 5'b10000: //LUI指令将立即数左移16位
    rdata1[4:0]; //可变目标左右移量为rs[4:0]
```

1.7 寄存器写使能信号

在这一部分中定义一个中间控制信号 mov-con, 利用控制 movn 和 movz, 来简化判断

```
//定义一个中间控制信号,用于控制movn和movz
wire mov_con;
```

```

assign mov_con =(rdata2==32'b0);
//定义寄存器写使能信号
assignRegWrite = (opcode==`R_TYPE&&func==`JR)? 0: //JR时不写入
    ((opcode ==`R_TYPE&& {func[5:3],func[1]}!=4'b0011)||opcode[5:3]==3'b001||MemRead)? 1:
    (opcode==`R_TYPE &&func==`MOVZ)? mov_con: //rt==0则写入
    (opcode==`R_TYPE &&func==`MOVN)? ~mov_con: //rt!=0则写入
    (opcode==`JAL)? 1: //JAL的情况,拉高wen
    0; //其他情况下不写入

```

1.8 分支和跳转的地址

这是本模块的最后部分,用于计算分支和跳转的地址,主要是针对 Jump 指令和分支指令,具体操作严格按照 MIPS 指令集手册的定义,其实这一部分放在 ex.v 中也可以

```

//计算出分支目标地址
wire [15:0]offset_temp;
assign offset_temp=offset<<2;
assign Branch_addr={{16{offset_temp[15]}},offset_temp};

//计算出跳转目标地址
assign Jump_addr = (opcode[5:1]==5'b00001)?{PC[31:28],instr_index,2'b00}://J型指令
    rdata1; //R型跳转指令

```

2 执行

由于主要的工作都在译码模块中,所以执行模块的设计相对简单,主要是实例化 ALU 和 Shifter,另外还有一部分是分支控制信号的产生,主要就是对比大小的跳转指令利用 ALU-result 作为分支控制信号,以及 BEQ 和 BNE 指令分别利用 Zero 和 Zero 作为分支控制信号。

```

//在Shifter和ALU的结果中二选一,输出一个结果
assign Result = (AluShi_sel)? Shifter_result:
    ALU_result;

//对于分支指令,将其分支控制信号赋值
wire ALU_Branch_temp;
assign ALU_Branch_temp = (opcode[5:1]==5'b00011 || opcode==`REGIMM)? ALU_result :
    //对于比大小的分支指令
    (opcode==`BEQ)? Zero : //对于beq,
        两个相等,跳转
    (opcode==`BNE)? ~Zero : //对于bne,
        两个不相等,跳转
    0; //其他一般情况下,
        不跳转

assign ALU_Branch = ALU_Branch_temp;

```


3 计数

PC 模块主要是对 PC 进行加 4, 以及对分支和跳转指令的处理,PC 的设计同样简单, 但值得注意的是要 $PC \leq PC + \text{Branch_addr} + 4$ 来流出分支延迟槽, 虽然在目前的单周期以及多周期中没有作用, 但是在流水线中分支延迟槽是不可或缺的, 另外,PC 也是本次实验中除了之前写的 reg-file 以外唯一的时序逻辑。

```
wire Branch_f;
assign Branch_f = Branch && ALU_Branch;

always @(posedge clk) begin
    if(rst) begin
        PC <= 32'd0; //复位 → CPU 从头开始执行
    end
    else if(Branch_f)begin //对于分支,分支地址=当前PC+偏移+4
        PC <= PC + Branch_addr + 4;
    end
    else if(Jump)begin //对于跳转
        PC <= Jump_addr;
    end
    else begin //其他一般情况
        PC <= PC + 4;
    end
end
end
```

4 移位

shifter 模块中, 利用 shifterop 来对 op-A 进行不同的操作, 分为左移, 逻辑左移以及逻辑右移, 在设计 shifter 的时候, 利用了 signed 来进行无符号数和有符号数的转换

```
/*
assign Result = (Shiftop == 2'b00) ? (A << B) : // 逻辑左移
                (Shiftop == 2'b10) ? (A >> B) : // 逻辑右移
                (Shiftop == 2'b11) ? ($signed(A) >>> B) : // 算术右移
                A; // 默认输出 A
*/
wire signed [`DATA_WIDTH - 1:0] A_signed;
assign A_signed = $signed(A);

assign Result = (Shiftop == 2'b00) ? (A << B) : // 逻辑左移
                (Shiftop == 2'b10) ? (A >> B) : // 逻辑右移
                (Shiftop == 2'b11) ? ($unsigned(A_signed) >>> B) : // 算术右移
                A;
```

注释里面的部分为错误代码, 主要原因是 wire 直接定义一个无符号数, 在这里会出现无符号数与有符号数的转换错误。

5 simple-cpu

这一部分主要是各个线的连接以及实例化。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

在本次实验中,主要出现了 shifter 算术右移问题,在进行行为逻辑仿真测试的时候,发现逻辑右移无论如何不能产生一个正确的结果,于是进行了分别的测试,原始未修改的代码如下

```
assign Result = (Shiftop == 2'b00) ? (A << B) : // 逻辑左移
               (Shiftop == 2'b10) ? (A >> B) : // 逻辑右移
               (Shiftop == 2'b11) ? ($signed(A) >>> B) : // 算术右移
               A; // 默认输出 A
```

推测是有符号数与无符号数转化出现问题,故需要人为进行转化。

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	算数右移	shifter	无法通过测试样例	有符号数和无符号数转化出现问题	进行人为转化

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

掌握单周期 CPU 模块化分层设计思想,熟悉取指、译码、执行、访存、写回完整工作流程,能够根据功能划分独立子模块,区分时序逻辑与组合逻辑设计差异,摒弃软件顺序思维,建立硬件并行电路设计思维。熟练掌握标准化、可综合 Verilog 代码编写规范,能够独立完成模块例化、信号互联、位域拼接、多条件逻辑设计,熟练解决非对齐访存、字节掩码控制、地址对齐等复杂硬件逻辑问题。代码结构冗余为实现复杂访存逻辑,大量使用多层三元运算符嵌套,代码层级繁杂、可读性较差,存在少量冗余逻辑,不利于后期迭代优化。

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- 思考题 1: ALUOp 的编码有什么规律?表格中的 ALUOp 编码是否还有优化空间?。对于 R-type 的指令,

1.6.3 ALUOp 编码



R-Type 指令	func (6-bit)	ALUOp (3-bit)	ALUOp 编码	opcode (6-bit)	I-Type 指令	ALUOp 编码
add	10 00 00	010	ADD/SUB: func[3:2] == 2'b00	001 0 00	addi	ADD: opcode[2:1] == 2'b00 ALUOp = {opcode[1], 2'b10}
addu	10 00 01			001 0 01	addiu	
sub	10 00 10	110	ALUOp = {func[1], 2'b10}			
subu	10 00 11					
and	10 01 00	000	逻辑运算: func[3:2] == 2'b01	001 1 00	andi	逻辑运算: opcode[2] == 1'b1 ALUOp = {opcode[1], 1'b0, opcode[0]}
or	10 01 01	001		001 1 01	ori	
xor	10 01 10	100	ALUOp = {func[1], 1'b0, func[0]}	001 1 10	xori	非运算类指令, 需单独处理
nor	10 01 11	101		001 1 11	lui	
slt	10 10 10	111	比较运算: func[3:2] == 2'b10 ALUOp = {~func[0], 2'b11}	001 0 10	slti	比较运算: opcode[2:1] == 2'b01 ALUOp = {~opcode[0], 2'b11}
sltu	10 10 11	011		001 0 11	sltiu	

- 新增3个ALUOp, 分别对应XOR、NOR和SLTU操作(表格中橙色字)
- 可以改变实验项目1中已实现的5种操作编码
- 思考题: ALUOp的编码有什么规律?表格中的ALUOp编码是否有优化空间?

对于同一类运算(加减法, 逻辑运算, 比大小), 他们的 ALUOp 和自己的 func 是有相同位的, 于是可以采用判断 func 再利用拼接来产生 ALUOp, 另外对于 I-type 指令也是类似的规律, 此外对于 R 型和 I 型指令, 他们的拼

接过滤是完全相同的, 差别是一个用 opcode, 一个用 func. 我们观察到 R 和 I 的信号产生差别只有 opcode 和 func, 是否存在将通以运算的 R 和 I 型指令合并的可能? 另外目前 ADD/SUB 和逻辑运算的 ALUop 编码中, 部分位的使用可能存在冗余。例如 ADD/SUB 的 func[1],2'b10 中可以检查是否有进一步压缩的可能性。

五、 实验时长

在课后,你花费了大约 20 小时完成此次实验,其中各个阶段的用时如表 2所示。

步骤	时长(小时)	说明
设计	5	确定自己将 cpu 分成几个小模块,并且大致确定每一部分要实现的功能
开发	10	完成代码的编写
调试	5	主要在 debug,借助实验平台还有 ai 工具的帮助

表 2: 实验时长